



# CS244 Advanced Programming Applications

Dr Walid M. Aly

## Lecture 1

# Revision on Basic Java Concepts



## TIOBE Programming Community Index for December 2011

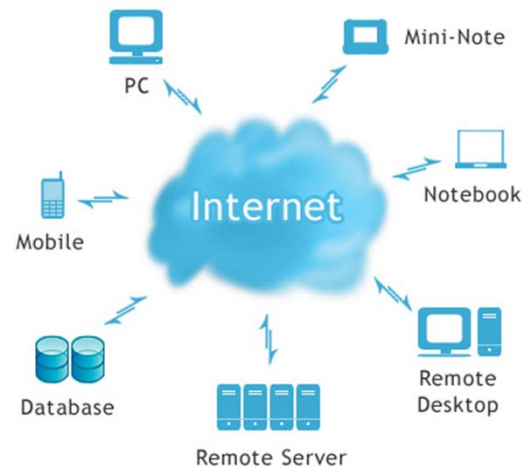
<http://www.tiobe.com>

| Position<br>Dec 2011 | Position<br>Dec 2010 | Programming<br>Language | Ratings<br>Dec 2011 |
|----------------------|----------------------|-------------------------|---------------------|
| 1                    | 1                    | Java                    | 17.561%             |
| 2                    | 2                    | C                       | 17.057%             |
| 3                    | 3                    | C++                     | 8.252%              |
| 4                    | 5                    | C#                      | 8.205%              |
| 5                    | 8                    | Objective-C             | 6.805%              |
| 6                    | 4                    | PHP                     | 6.001%              |
| 7                    | 7                    | (Visual) Basic          | 4.757%              |
| 8                    | 6                    | Python                  | 3.492%              |
| 9                    | 9                    | Perl                    | 2.472%              |
| 10                   | 12                   | JavaScript              | 2.199%              |
| 11                   | 11                   | Ruby                    | 1.494%              |
| 12                   | 10                   | Delphi/Object Pascal    | 1.245%              |
| 13                   | 13                   | Lisp                    | 1.175%              |



# What is new with Java?

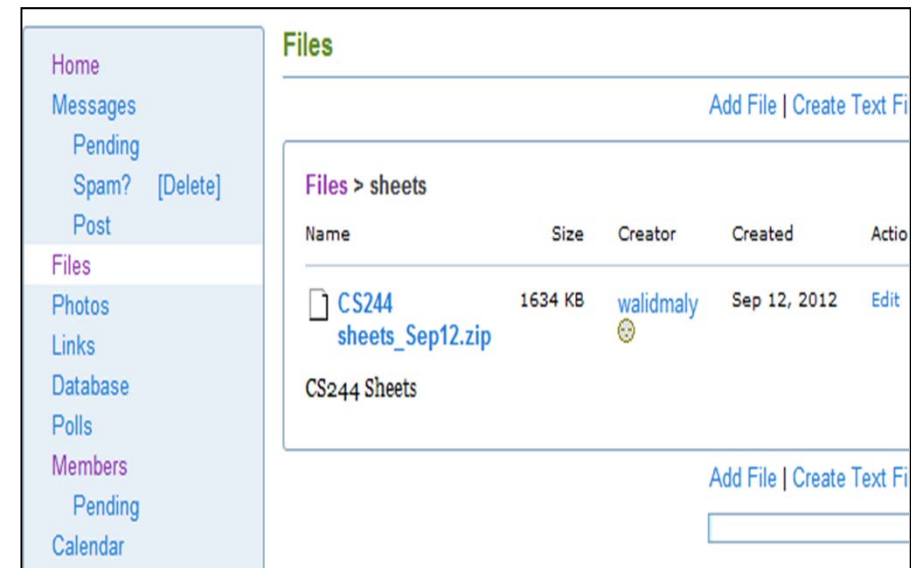
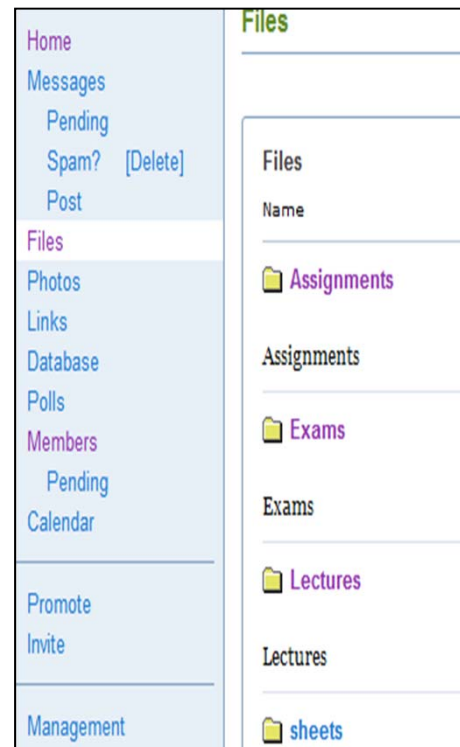
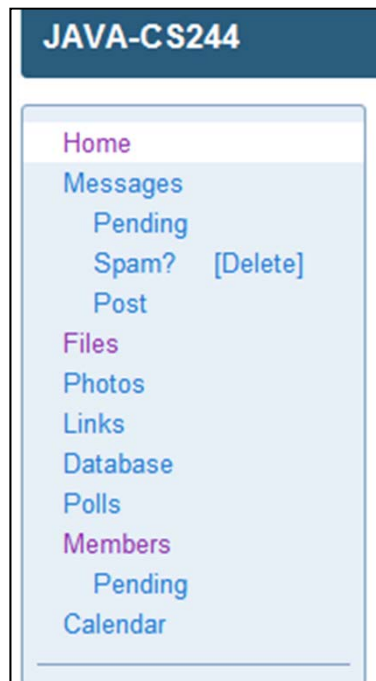
- Java 7 Released July 2011
- Java & the Cloud Computing
- Java & Android





# Course Mail Group

<http://groups.yahoo.com/group/JAVA-CS244/>





# Installing NetBeans

<http://netbeans.org/downloads/index.html>



| NetBeans IDE Download Bundles              |                          |                          |                          |                          |                          |
|--------------------------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|
| Supported technologies *                   | Java SE                  | Java EE                  | C/C++                    | PHP                      | All                      |
| NetBeans Platform SDK                      | •                        | •                        |                          |                          | •                        |
| Java SE                                    | •                        | •                        |                          |                          | •                        |
| Java FX                                    | •                        | •                        |                          |                          | •                        |
| Java EE                                    |                          | •                        |                          |                          | •                        |
| Java ME                                    |                          |                          |                          |                          | •                        |
| Java Card™ 3 Connected                     |                          |                          |                          |                          | •                        |
| C/C++                                      |                          |                          | •                        |                          | •                        |
| Groovy                                     |                          |                          |                          |                          | •                        |
| PHP                                        |                          |                          |                          | •                        | •                        |
| Bundled servers                            |                          |                          |                          |                          |                          |
| GlassFish Server Open Source Edition 3.1.1 |                          | •                        |                          |                          | •                        |
| Apache Tomcat 7.0.22                       |                          | •                        |                          |                          | •                        |
|                                            | <a href="#">Download</a> | <a href="#">Download</a> | <a href="#">Download</a> | <a href="#">Download</a> | <a href="#">Download</a> |
|                                            | Free, 72 MB              | Free, 166 MB             | Free, 49 MB              | Free, 47 MB              | Free, 249 MB             |



## JDK7 & NetBeans Bundle

### JDK 7u3 with NetBeans 7.1



<http://www.oracle.com/technetwork/java/javase/downloads/jdk-7-netbeans-download-432126.html>



## objects and classes

- We work with **physical objects** all the time: televisions, cars, customers, reports, light bulbs, printers.
- A software *object* is a representation of a thing that we want to manipulate in our program.
- A class is a **template** that defines what an object's data and methods will be.
- Objects share two characteristics: **state** and **behavior**.
- The **state** of an object is represented by **variables** and **behavior** by a set of **methods**
- **OOP reflects how people think of real world.**

```
class className
{
    Variables describing state
    Methods describing behavior
}
```

```
class Circle
{
    double radius;
    .....
    double get Area()
    {
        .....
    }
    .....
}
```

```
class Student
{
    String name;
    double GPA;
    .....
    void registerUnit
    {
        .....
    }
    .....
}
```

Student object 1

Name =Ahmed Aly  
Registration number=245245  
GPA=3.2

register a unit

Student object 2

Name =Mohamed Fathi  
Registration number=34345  
GPA=2.2

register a unit

# A Simple Java class

```
// author: Ahmed Ali
public class Car{
    String color;
    private int speed;
    private int numOfDoors=4;
    public void incrementSpeed(int value){
        .....
    }
    public void decrementSpeed(int value){
        .....
    }
    public int getSpeed(){
        .....
    }
    public boolean isMoving()
    {
        .....
    }
    public static void getNumberOfCars()
    {.....}
}
}
```



# Java classes

- Classes in your program might be your own classes or classes that already exist in java API.
- A Java class has static and non static variables
- Java class has static and non static methods(instance members)
- Java class has one or more Constructors
- Constructors are special type of methods , the **must** have the same name as class , and with no return type.
- We use constructor to create objects from classes
- Variables, methods and constructors are known as **class members**

|                             |
|-----------------------------|
| class className             |
| {                           |
| Variables describing state  |
| Constructors                |
| Methods describing behavior |
| }                           |

```
public class A
{
    public static int x;
    public int y;
    public A()
    {
        .....
    }
    public A(int z)
    {
        .....
    }
    public static void m1()
    {
        .....
    }

    public void m2()
    {
        .....
    }
}
```





# How to use the class Members

➤ Static members are accessed by the name of class  
ex : `A.x; A.m1();`

➤ To use non static members you have to create objects from the class using constructor  
ex: `A a1 = new A();`  
and then access non static members with the object you created  
ex : `a1.y; a1.m2();`

```
public class Test{
    public static void main (String [] arg){
        //access static members
        A.m1();
        int i=A.x;
        System.out.println("x="+i);
        //access instance members
        A a1=new A();
        a1.m2();
        int j=a1.y;
        System.out.println("y="+j);
    }
}
```

```
public class A{
    public static int x=10;
    public int y=20;
    public A()
    {
        System.out.println("Creating Object");
    }
    public A(int z)
    {
        System.out.println("Creating Object with z="+z);
    }
    public static void m1()
    {
        System.out.println("Hi from m1");
    }
    public void m2()
    {
        System.out.println("Hi from m2");
    }
}
```

```
Hi from m1
x=10
Creating Object
Hi from m2
y=20
Press any key to continue . . .
```



## Example :Class Radio

```
public class Radio{  
    int volume;  
    String station;  
    boolean powerStatus;  
  
    public void increaseVolume(){  
        volume++;  
    }  
    public void decreaseVolume(){  
        volume--;  
    }  
    public void changeStation(String newStation){  
        station=newStation;  
    }  
    public void turnOn(){  
        volume=3;  
        powerStatus=true;  
    }  
    public void turnOff(){  
        volume=0;  
        powerStatus=false;  
    }  
}
```

Radio object1



volume =4  
station="Negoom FM"  
Powerstatus=true

Radio object2

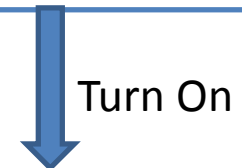


volume =7  
station="Quran"  
Powerstatus=true

Radio object3



volume =0  
station="FM"  
Powerstatus=false



Turn On

volume =3  
station="FM"  
Powerstatus=true



# Constructors

A class provides methods of a special type, known as *constructors*, which are used to create a new object (instance).

- Constructor must have the **same** name as the class name.
- Constructors do not have a return type—not even void, can not be static.
- *default blank constructor, is provided automatically only if no constructors are explicitly declared in the class.*
- A constructor is generally used to do initial setup for an object

```
class Circle {  
    double radius=0 ;  
    public Circle()  
    {  
  
    }  
    public Circle (double r)  
    {  
        radius=r;  
    }  
    public double findArea()  
    {  
        //  
    }  
}
```

multiple constructors with the same name but different signatures can exist.(**overloading**)



## How to create an object from class?

- To construct an object from a class, invoke a constructor of the class using the **new** operator, as follows: `new ClassName(arguments);`
- constructor can contain any type of code that a normal method could contain. *When constructor is invoked an object is created and also code in constructor is executed.*

```
class Circle {  
    double radius=0 ;  
    public Circle()  
    {  
  
    }  
    public Circle (double r)  
    {  
        radius=r;  
    }  
    public double findArea()  
    {  
        return radius * radius * 3.14;  
    }  
}
```

```
class Test{  
    public static void main (String [] arg)  
    {  
        Circle c1;  
        c1=new Circle();  
        Circle c2= new Circle();  
        Circle c3= new Circle(5.1);  
    }  
}
```

c1,c2,c3 are reference data types

c1 → radius=0

c2 → radius=0

c3 → radius=5.1

## Example :Adding constructors to class Radio

```
public class Radio{
    int volume;
    String station;
    boolean powerStatus;
    public Radio()
    {
        turnOn();
        Volume=3;
    }
    public Radio(int startVolume)
    {
        turnOn();
        Volume= startVolume;
    }
    public void increaseVolume(){
        volume++;
    }
    public void decreaseVolume(){
        volume--;
    }
    public void changeStation(String newStation){
        station=newStation;}
}
```

```
public void turnOn(){
    powerStatus=true;
}
public void turnOff(){
    volume=0;
    powerStatus=false;
}
}
```

```
Public class Test
{
    public static void main (String [] arg)
    Radio radio1=new Radio();
    Radio radio2=new Radio(2);
    radio2.increaseVolume();
    .....
}
```



# Variables Initialization

**Member variables** that are declared but not initialized will be set to a default by the compiler

| Data Type                        | Default Value |
|----------------------------------|---------------|
| Byte-short-int-long-float-double | 0             |
| char                             | '\u0000'      |
| String (or any object reference) | null          |
| boolean                          | false         |

**local variables** :that are declared but not initialized will not be set to a default by the compiler  
Accessing an uninitialized local variable will result in a compile-time error



## Accessing an Object's instance variables and Methods

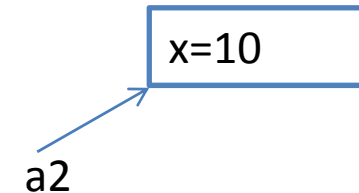
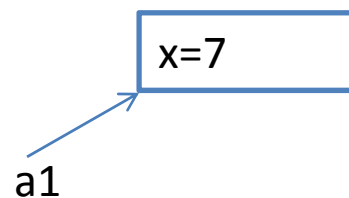
```
class A
{
    public int y;
    public int x;
    public void m(){

    }
    public void m2(){
        y=9;
        m();
    }
}
```

```
class Test {
    public static void main (String [] arg)
    {
        A a1=new A();
        a1.x=7;
        a1.m2();
        A a2=new A();
        a2.x=10;
        a2.m2    ();

    }
}
```

a1,a2 are of **reference** data type



### Rules :

- Instance variable of object are accessed from another class by : **objectName.variableName**
- Instance methods are accessed from another class by : **objectName.methodName(argument)**
- Inside same class , members can be called with their names only

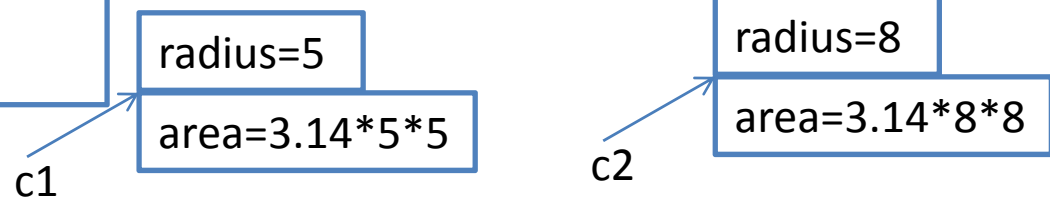


## Example :Accessing members of class Circle

```
class Circle{
public int radius;
public double area;
public void setArea()
{
this.area=3.14*(this.radius)*(this.radius);
}
Circle()
{
}
Circle(int r)
{
this.radius=r;
}
}
```

```
class Test {
public static void main (String [] arg)
{
Circle c1=new Circle();
c1.radius=5;
c1.setArea();
System.out.println(c1.area);
Circle c2=new Circle(8);
c2.setArea();
System.out.println(c2.area);
}
}
```

Rule :



➤ Instance variables in instance methods are always related to the calling object, JAVA add **this.** before each of them implicitly

➤ You can omit writing “this”





## Example :Accessing members of class BankAccount

### Class BankAccount

```
public class BankAccount
{
    int accountNum=0;
    float accountBalance=0;
    public BankAccount( ) {}
    public BankAccount(int accountNumber, float balance)
    {
        accountNum=accountNumber;
        accountBalance=balance;
    }
    void setAccountNumber(int num)
    {
        accountNum = num;
    }
    void withdraw(float amount)
    {
        accountBalance = accountBalance -amount;
    }
    void deposit (float amount)
    {
        accountBalance = accountBalance + amount;
    }
}
```

### Class Bank

```
public class Bank
{
    public static void main (String [] arg)
    {
        BankAccount account1=new BankAccount();
        account1.setAccountNumber(456);
        account1.deposit(1000);
        BankAccount account2=new BankAccount(100,200);
    }
}
```



# Static Class members: **Static variables**

## Static variables (class Variables)

Ex: **static** int x;

- Only one copy of this variable exist, all objects share the same copy with the same value.
- Can be accessed without creation of any object.
- Can be accessed in same class by **variableName** or **classname.variableName** or **ObjectName.variableName**.
- Can be accessed in another class by **classname.variableName** or **ObjectName.variableName**.

**Local Variables can never be declared static.**

```
class Student{
    static String dean;
    String name;
    int registrationNumber;
    Student( String studentName , int regNumber)
    {
        name=studentName;
        registrationNumber=regNumber;
    }
}
```

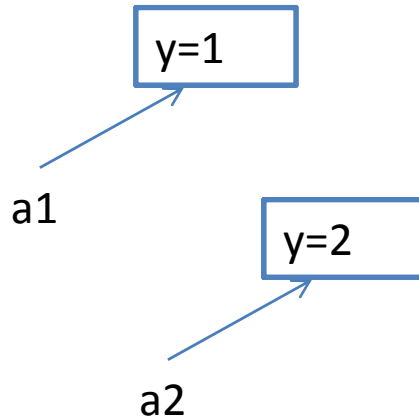
```
class Test{
    public static void main (String [] arg)
    {
        Student.dean="Dr Mohamed";
        Student student1=new Student("Ahmed Aly", 123);
        Student student2=new Student("Akram ahmed", 367);
        System.out.println(student1.dean);
        System.out.println(student2.dean);
    }
}
```



# Example

```
class A{  
    static int x;  
    int y;  
}
```

x 3



```
class Test  
{  
    public static void main (String [] arg)  
    {  
        A.x++;  
        A a1=new A();  
        a1.x++;  
        a1.y++;  
        A a2=new A();  
        a2.x++;  
        a2.y++;  
        a2.y++;  
    }  
}
```



# Static Class members :Static methods

## Static methods

Ex: `static void m(){}`

- Can be called without creation of any object
- **Can only access static variables and only call static methods.**
- Can be called in same class by `methodName()` or `classname.methodName()` or `ObjectName.methodName()` (same effect)
- Can be called in another class by `classname.methodName()` or `ObjectName.methodName()` (same effect)

**access modifiers and static keyword can appear in any order**

```
class Student{
    static String deanName;
    String name;
    int registrationNumber;
    Student( String studentName , int regNumber){
        name=studentName;
        registrationNumber=regNumber;
    }
    public static void setDeanName(String name)
    {
        deanName=name;
    }
}
```

```
class Test{
    public static void main (String [] arg){
        Student.setDeanName("Dr Mohamed");
        Student student1=new Student("Ahmed Aly", 123);
        Student student2=new Student("Akram ahmed", 367);
    }
}
```



# Your role

- this keyword
- passing by value and passing by reference
- Array
- UML class diagram